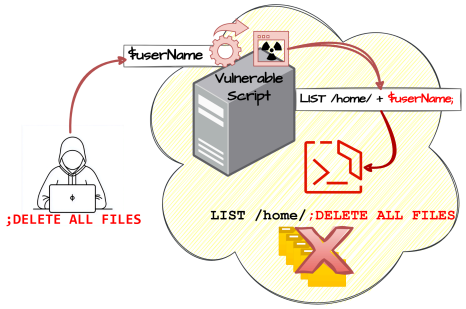
CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

Weakness ID: 78
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product constructs all or part of an OS command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended OS command when it is sent to a downstream component.



Extended Description

This weakness can lead to a vulnerability in environments in which the attacker does not have direct access to the operating system, such as in web applications. Alternately, if the weakness occurs in a privileged program, it could allow the attacker to specify commands that normally would not be accessible, or to call alternate commands with privileges that the attacker does not have. The problem is exacerbated if the compromised process does not follow the principle of least privilege, because the attacker-controlled commands may run with special system privileges that increases the amount of damage.

There are at least two subtypes of OS command injection:

- The application intends to execute a single, fixed program that is under its own control. It intends to use externally-supplied inputs as arguments to that program. For example, the program might use system("nslookup [HOSTNAME]") to run nslookup and allow the user to supply a HOSTNAME, which is used as an argument. Attackers cannot prevent nslookup from executing. However, if the program does not remove command separators from the HOSTNAME argument, attackers could place the separators into the arguments, which allows them to execute their own program after nslookup has finished executing.
- The application accepts an input that it uses to fully select which program to run, as well as which commands to use. The application simply redirects this entire command to the operating system. For example, the program might use "exec([COMMAND])" to execute the [COMMAND] that was supplied by the user. If the COMMAND is under attacker control, then the attacker can execute arbitrary commands or programs. If the command is being executed using functions like exec() and CreateProcess(), the attacker might not be able to combine multiple commands together in the same line.

From a weakness standpoint, these variants represent distinct programmer errors. In the first variant, the programmer clearly intends that input from untrusted parties will be part of the arguments in the command to be executed. In the second variant, the programmer does not intend for the command to be accessible to any untrusted party, but the programmer probably has not accounted for alternate ways in which malicious attackers can provide input.

Alternate Terms

Shell injection	
Shell metacharacters	
OS Command Injection	

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands; DoS: Crash, Exit, or Restart; Read Files or Directories; Modify Files or Directories; Read Application Data; Modify Application Data; Hide Activities	**Scope: Confidentiality, Integrity, Availability, Non-Repudiation** Attackers could execute unauthorized operating system commands, which could then be used to disable the product, or read and modify data for which the attacker does not have permissions to access directly. Since the targeted application is directly executing the commands instead of the attacker, any malicious activities may appear to come from the application or the application's owner.

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	If at all possible, use library calls rather than external processes to recreate the desired functionality.
Architecture and Design; Operation	**Strategy: *Sandbox or Jail*** Run the code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by the software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows the software to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of the application may still be subject to compromise. Be careful to avoid CWE-243 and other weaknesses related to jails. **Effectiveness: Limited** **Note:** The effectiveness of this mitigation depends on the prevention capabilities of the specific sandbox or jail being used and might only help to reduce the scope of an attack, such as restricting the attacker to certain system calls or limiting the portion of the file system that can be accessed.
Architecture and Design	**Strategy: *Attack Surface Reduction*** For any data that will be used to generate a command to be executed, keep as much of that data out of external control as possible. For example, in web applications, this may require storing the data locally in the session's state instead of sending it out to the client in a hidden form field.
Architecture and Design	For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.
Architecture and Design	**Strategy: *Libraries or Frameworks*** Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, consider using the ESAPI Encoding control [REF-45] or a similar tool, library, or framework. These will help the programmer encode outputs in a manner less prone to error.
Implementation	**Strategy: *Output Encoding*** While it is risky to use dynamically-generated query strings, code, or commands that mix control and data together, sometimes it may be unavoidable. Properly quote arguments and escape any special characters within those arguments. The most

	conservative approach is to escape or filter all characters that do not pass an extremely strict allowlist (such as everything that is not alphanumeric or white space). If some special characters are still needed, such as white space, wrap each argument in quotes after the escaping/filtering step. Be careful of argument injection (CWE-88).
Implementation	If the program to be executed allows arguments to be specified within an input file or from standard input, then consider using that mode to pass arguments instead of the command line.
Architecture and Design	**Strategy:** ***Parameterization*** If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Some languages offer multiple functions that can be used to invoke commands. Where possible, identify any function that invokes a command shell using a single string, and replace it with a function that requires individual arguments. These functions typically perform appropriate quoting and filtering of arguments. For example, in C, the system() function accepts a string that contains the entire command to be executed, whereas execl(), execve(), and others require an array of strings, one for each argument. In Windows, CreateProcess() only accepts one command at a time. In Perl, if system() is provided with an array of arguments, then it will quote each of the arguments.
Implementation	**Strategy:** ***Input Validation*** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When constructing OS command strings, use stringent allowlists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping. Note that proper output encoding, escaping, and quoting is the most effective solution for preventing OS command injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent OS command injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, when invoking a mail program, you might need to allow the subject field to contain otherwise-dangerous inputs like ";" and ">" characters, which would need to be escaped or otherwise handled. In this case, stripping the character might reduce the risk of OS command injection, but it would produce incorrect behavior because the subject field would not be recorded as the user intended. This might seem to be a minor inconvenience, but it could be more important when the program relies on well-structured subject lines in order to pass messages to other components. Even if you make a mistake in your validation (such as forgetting one out of 100 input fields), appropriate encoding is still likely to protect you from injection-based attacks. As long as it is not done in isolation, input validation is still a useful technique, since it may significantly reduce your attack surface, allow you to detect some attacks, and provide other security benefits that proper encoding does not address.
Architecture and Design	**Strategy:** ***Enforcement by Conversion*** When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the

	actual filenames or URLs, and reject all other inputs.
Operation	**Strategy:** ***Compilation or Build Hardening*** Run the code in an environment that performs automatic taint propagation and prevents any command execution that uses tainted variables, such as Perl's "-T" switch. This will force the program to perform validation steps that remove the taint, although you must be careful to correctly validate your inputs so that you do not accidentally mark dangerous inputs as untainted (see CWE-183 and CWE-184).
Operation	**Strategy:** ***Environment Hardening*** Run the code in an environment that performs automatic taint propagation and prevents any command execution that uses tainted variables, such as Perl's "-T" switch. This will force the program to perform validation steps that remove the taint, although you must be careful to correctly validate your inputs so that you do not accidentally mark dangerous inputs as untainted (see CWE-183 and CWE-184).
Implementation	Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended). The messages should not reveal the methods that were used to determine the error. Attackers can use detailed information to refine or optimize their original attack, thereby increasing their chances of success. If errors must be captured in some detail, record them in log messages, but consider what could occur if the log messages can be viewed by attackers. Highly sensitive information such as passwords should never be saved to log files. Avoid inconsistent messaging that might accidentally tip off an attacker about internal state, such as whether a user account exists or not. In the context of OS Command Injection, error information passed back to the user might reveal whether an OS command is being executed and possibly which command is being used.
Operation	**Strategy:** ***Sandbox or Jail*** Use runtime policy enforcement to create an allowlist of allowable commands, then prevent use of any command that does not appear in the allowlist. Technologies such as AppArmor are available to do this.
Operation	**Strategy:** ***Firewall*** Use an application firewall that can detect attacks against this weakness. It can be beneficial in cases in which the code cannot be fixed (because it is controlled by a third party), as an emergency prevention measure while more comprehensive software assurance measures are applied, or to provide defense in depth [REF-1481]. **Effectiveness: Moderate** **Note:** An application firewall might not cover all possible input vectors. In addition, attack techniques might be available to bypass the protection mechanism, such as using malformed inputs that can still be processed by the component that receives those inputs. Depending on functionality, an application firewall might inadvertently reject or modify legitimate requests. Finally, some manual effort may be required for customization.
Architecture and Design; Operation	**Strategy:** ***Environment Hardening*** Run your code using the lowest privileges that are required to accomplish the necessary tasks [REF-76]. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations.
Operation; Implementation	**Strategy:** ***Environment Hardening*** When using PHP, configure the application so that it does not use register_globals. During implementation, develop the application so that it does not rely on this feature, but be wary of implementing a register_globals emulation that is subject to weaknesses such as CWE-95, CWE-621, and similar issues.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	C	77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
CanAlsoBe	B	88	Improper Neutralization of Argument Delimiters in a Command ('Argument Injection')
CanFollow	B	184	Incomplete List of Disallowed Inputs

Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	C	137	Data Neutralization Issues

Relevant to the view "Weaknesses for Simplified Mapping of Published Vulnerabilities" (View-1003)

Nature	Type	ID	Name
ChildOf	C	74	Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')

Relevant to the view "Architectural Concepts" (View-1008)

Nature	Type	ID	Name
MemberOf	C	1019	Validate Inputs

Relevant to the view "CISQ Quality Measures (2020)" (View-1305)

Nature	Type	ID	Name
ChildOf	C	77	Improper Neutralization of Special Elements used in a Command ('Command Injection')

Relevant to the view "CISQ Data Protection Measures" (View-1340)

Nature	Type	ID	Name
ChildOf	C	77	Improper Neutralization of Special Elements used in a Command ('Command Injection')

Modes Of Introduction

Phase	Note
Implementation	REALIZATION: This weakness is caused during implementation of an architectural security tactic.

Applicable Platforms

Languages	Class: Not Language-Specific *(Undetermined Prevalence)*
Technologies	Class: Not Technology-Specific *(Undetermined Prevalence)* AI/ML *(Undetermined Prevalence)* Web Server *(Often Prevalent)*

Likelihood Of Exploit

High

Demonstrative Examples

Example 1

This example code intends to take the name of a user and list the contents of that user's home directory. It is subject to the first variant of OS command injection.

Example Language: **PHP** *(bad code)*

```
$userName = $_POST["user"];
$command = 'ls -l /home/' . $userName;
system($command);
```

The $userName variable is not checked for malicious input. An attacker could set the $userName variable to an arbitrary OS command such as:

(attack code)

```
;rm -rf /
```

Which would result in $command being:

(result)

```
ls -l /home/;rm -rf /
```

Since the semi-colon is a command separator in Unix, the OS would first execute the ls command, then the rm command, deleting the entire file system.

Also note that this example code is vulnerable to Path Traversal (CWE-22) and Untrusted Search Path (CWE-426) attacks.

Example 2

The following simple program accepts a filename as a command line argument and displays the contents of the file back to the user. The program is installed setuid root because it is intended for use as a learning tool to allow system administrators in-training to inspect privileged system files without giving them the ability to modify them or damage the system.

Example Language: **C** (bad code)

```
int main(int argc, char** argv) {
  char cmd[CMD_MAX] = "/usr/bin/cat ";
  strcat(cmd, argv[1]);
  system(cmd);
}
```

Because the program runs with root privileges, the call to system() also executes with root privileges. If a user specifies a standard filename, the call works as expected. However, if an attacker passes a string of the form ";rm -rf /", then the call to system() fails to execute cat due to a lack of arguments and then plows on to recursively delete the contents of the root partition.

Note that if argv[1] is a very long argument, then this issue might also be subject to a buffer overflow (CWE-120).

Example 3

This example is a web application that intends to perform a DNS lookup of a user-supplied domain name. It is subject to the first variant of OS command injection.

Example Language: **Perl** (bad code)

```
use CGI qw(:standard);
$name = param('name');
$nslookup = "/path/to/nslookup";
print header;
if (open($fh, "$nslookup $name|")) {
  while (<$fh>) {
    print escapeHTML($_);
    print "<br>\n";
  }
  close($fh);
}
```

Suppose an attacker provides a domain name like this:

(attack code)

```
cwe.mitre.org%20%3B%20/bin/ls%20-l
```

The "%3B" sequence decodes to the ";" character, and the %20 decodes to a space. The open() statement would then process a string like this:

(result)

```
/path/to/nslookup cwe.mitre.org ; /bin/ls -l
```

As a result, the attacker executes the "/bin/ls -l" command and gets a list of all the files in the program's working directory. The input could be replaced with much more dangerous commands, such as installing a malicious program on the server.

Example 4

The example below reads the name of a shell script to execute from the system properties. It is subject to the second variant of OS command injection.

Example Language: **Java** *(bad code)*

```
String script = System.getProperty("SCRIPTNAME");
if (script != null)
  System.exec(script);
```

If an attacker has control over this property, then they could modify the property to point to a dangerous program.

Example 5

In the example below, a method is used to transform geographic coordinates from latitude and longitude format to UTM format. The method gets the input coordinates from a user through a HTTP request and executes a program local to the application server that performs the transformation. The method passes the latitude and longitude coordinates as a command-line option to the external program and will perform some processing to retrieve the results of the transformation and return the resulting UTM coordinates.

Example Language: **Java** *(bad code)*

```
public String coordinateTransformLatLonToUTM(String coordinates)
{
  String utmCoords = null;
  try {
    String latlonCoords = coordinates;
    Runtime rt = Runtime.getRuntime();
    Process exec = rt.exec("cmd.exe /C latlon2utm.exe -" + latlonCoords);
    // process results of coordinate transform

    // ...
  }
  catch(Exception e) {...}
  return utmCoords;
}
```

However, the method does not verify that the contents of the coordinates input parameter includes only correctly-formatted latitude and longitude coordinates. If the input coordinates were not validated prior to the call to this method, a malicious user could execute another program local to the application server by appending '&' followed by the command for another program to the end of the coordinate string. The '&' instructs the Windows operating system to execute another program.

Example 6

The following code is from an administrative web application designed to allow users to kick off a backup of an Oracle database using a batch-file wrapper around the rman utility and then run a cleanup.bat script to delete some temporary files. The script rmanDB.bat accepts a single command line parameter, which specifies what type of backup to perform. Because access to the database is restricted, the application runs the backup as a privileged user.

Example Language: **Java** *(bad code)*

```
...
String btype = request.getParameter("backuptype");
String cmd = new String("cmd.exe /K \"
  c:\\util\\rmanDB.bat "
  +btype+
  "&&c:\\utl\\cleanup.bat\"")

System.Runtime.getRuntime().exec(cmd);
...
```

The problem here is that the program does not do any validation on the backuptype parameter read from the user. Typically the Runtime.exec() function will not execute multiple commands, but in this case the program first runs the cmd.exe shell in order to run multiple commands with a single call to Runtime.exec(). Once the shell is invoked, it will happily execute multiple commands separated by two ampersands. If an attacker passes a string of the form "& del c:\\dbms*.*", then the application will execute this command along with the others specified by the program. Because of the nature of the application, it runs with the privileges necessary to interact with the database, which means whatever command the attacker injects will run with those privileges as well.

Example 7

The following code is a wrapper around the UNIX command cat which prints the contents of a file to standard out. It is also injectable:

Example Language: **C** *(bad code)*

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {

  char cat[] = "cat ";
  char *command;
  size_t commandLength;

  commandLength = strlen(cat) + strlen(argv[1]) + 1;
  command = (char *) malloc(commandLength);
  strncpy(command, cat, commandLength);
  strncat(command, argv[1], (commandLength - strlen(cat)) );

  system(command);
  return (0);
}
```

Used normally, the output is simply the contents of the file requested, such as Story.txt:

(informative)

```
./catWrapper Story.txt
```

(result)

```
When last we left our heroes...
```

However, if the provided argument includes a semicolon and another command, such as:

(attack code)

```
Story.txt; ls
```

Then the "ls" command is executed by catWrapper with no complaint:

(result)

```
./catWrapper Story.txt; ls
```

Two commands would then be executed: catWrapper, then ls. The result might look like:

(result)

```
When last we left our heroes...
Story.txt
SensitiveFile.txt
PrivateData.db
a.out*
```

If catWrapper had been set to have a higher privilege level than the standard user, arbitrary commands could be executed with that higher privilege.

Example 8

This example takes user input, passes it through an encoding scheme, then lists the contents of the user's home directory based on the user name.

Example Language: **Perl** (bad code)

```
sub GetUntrustedInput {
  return($ARGV[0]);
}

sub encode {
  my($str) = @_;
  $str =~ s/\&/\&amp;/gs;
  $str =~ s/\"/\&quot;/gs;
  $str =~ s/\'/\&apos;/gs;
  $str =~ s/\</\&lt;/gs;
  $str =~ s/\>/\&gt;/gs;
  return($str);
}

sub doit {
  my $uname = encode(GetUntrustedInput("username"));
  print "<b>Welcome, $uname!</b><p>\n";
  system("cd /home/$uname; /bin/ls -l");
}
```

The programmer attempts to encode dangerous characters, however the denylist for encoding is incomplete (CWE-184) and an attacker can still pass a semicolon, resulting in a chain with OS command injection (CWE-78).

Additionally, the encoding routine is used inappropriately with command execution. An attacker doesn't even need to insert their own semicolon. The attacker can instead leverage the encoding routine to provide the semicolon to separate the commands. If an attacker supplies a string of the form:

(attack code)

```
' pwd
```

then the program will encode the apostrophe and insert the semicolon, which functions as a command separator when passed to the system function. This allows the attacker to complete the command injection.

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2024-53899	Virtual environment builder does not correctly quote "magic" template strings, allowing OS command injection using a directory whose name contains shell metacharacters
CVE-2025-44844	file upload functionality in wireless access point allows OS command injection via shell metacharacters through the file name in a Content-Disposition header

CVE-2024-6091	Chain: AI agent platform does not restrict pathnames containing internal "/./" sequences (CWE-55), leading to an incomplete denylist (CWE-184) that does not prevent OS command injection (CWE-78)
CVE-2024-41316	Lua application in network device allows OS command injection into os.execute()
CVE-2024-44335	Chain: filter only checks for some shell-injection characters (CWE-184), enabling OS command injection (CWE-78)
CVE-2024-52803	Platform for handling LLMs has OS command injection during training due to insecure use of the "Popen" function
CVE-2020-10987	OS command injection in Wi-Fi router, as exploited in the wild per CISA KEV.
CVE-2020-10221	Template functionality in network configuration management tool allows OS command injection, as exploited in the wild per CISA KEV.
CVE-2020-9054	Chain: improper input validation (CWE-20) in username parameter, leading to OS command injection (CWE-78), as exploited in the wild per CISA KEV.
CVE-1999-0067	Canonical example of OS command injection. CGI program does not neutralize "|" metacharacter when invoking a phonebook program.
CVE-2001-1246	Language interpreter's mail function accepts another argument that is concatenated to a string used in a dangerous popen() call. Since there is no neutralization of this argument, both OS Command Injection (CWE-78) and Argument Injection (CWE-88) are possible.
CVE-2002-0061	Web server allows command execution using "|" (pipe) character.
CVE-2003-0041	FTP client does not filter "|" from filenames returned by the server, allowing for OS command injection.
CVE-2008-2575	Shell metacharacters in a filename in a ZIP archive
CVE-2002-1898	Shell metacharacters in a telnet:// link are not properly handled when the launching application processes the link.
CVE-2008-4304	OS command injection through environment variable.
CVE-2008-4796	OS command injection through https:// URLs
CVE-2007-3572	Chain: incomplete denylist for OS command injection
CVE-2012-1988	Product allows remote users to execute arbitrary commands by creating a file whose pathname contains shell metacharacters.

Weakness Ordinalities

Ordinality	Description
Primary	*(where the weakness exists independent of other weaknesses)*
Resultant	*(where the weakness is typically related to the presence of some other weaknesses)*

Detection Methods

Method	Details
Automated Static Analysis	This weakness can often be detected using automated static analysis tools. Many modern tools use data flow analysis or constraint-based techniques to minimize the number of false positives. Automated static analysis might not be able to recognize when proper input validation is being performed, leading to false positives - i.e., warnings that do not have any security consequences or require any code changes. Automated static analysis might not be able to detect the usage of custom API functions or third-party libraries that indirectly invoke OS commands, leading to false negatives - especially if the API/library code is not available for analysis. **Note:** This is not a perfect solution, since 100% accuracy and coverage are not feasible.

Automated Dynamic Analysis	This weakness can be detected using dynamic tools and techniques that interact with the product using large test suites with many diverse inputs, such as fuzz testing (fuzzing), robustness testing, and fault injection. The product's operation may slow down, but it should not become unstable, crash, or generate incorrect results. **Effectiveness: Moderate**
Manual Static Analysis	Since this weakness does not typically appear frequently within a single software package, manual white box techniques may be able to provide sufficient code coverage and reduction of false positives if all potentially-vulnerable operations can be assessed within limited time constraints. **Effectiveness: High**
Automated Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Bytecode Weakness Analysis - including disassembler + source code weakness analysis • Binary Weakness Analysis - including disassembler + source code weakness analysis **Effectiveness: High**
Dynamic Analysis with Automated Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Web Application Scanner • Web Services Scanner • Database Scanners **Effectiveness: SOAR Partial**
Dynamic Analysis with Manual Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Fuzz Tester • Framework-based Fuzzer **Effectiveness: SOAR Partial**
Manual Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Manual Source Code Review (not inspections) Cost effective for partial coverage: • Focused Manual Spotcheck - Focused manual analysis of source **Effectiveness: High**
Automated Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Source code Weakness Analyzer

	• Context-configured Source Code Weakness Analyzer **Effectiveness: High**
Architecture or Design Review	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Formal Methods / Correct-By-Construction Cost effective for partial coverage: • Inspection (IEEE 1028 standard) (can apply to requirements, design, source code, etc.) **Effectiveness: High**

Functional Areas

- Program Invocation

Affected Resources

- System Process

Memberships

Nature	Type	ID	Name
MemberOf	V	635	Weaknesses Originally Used by NVD from 2008 to 2016
MemberOf	C	714	OWASP Top Ten 2007 Category A3 - Malicious File Execution
MemberOf	C	727	OWASP Top Ten 2004 Category A6 - Injection Flaws
MemberOf	C	741	CERT C Secure Coding Standard (2008) Chapter 8 - Characters and Strings (STR)
MemberOf	C	744	CERT C Secure Coding Standard (2008) Chapter 11 - Environment (ENV)
MemberOf	C	751	2009 Top 25 - Insecure Interaction Between Components
MemberOf	C	801	2010 Top 25 - Insecure Interaction Between Components
MemberOf	C	810	OWASP Top Ten 2010 Category A1 - Injection
MemberOf	C	845	The CERT Oracle Secure Coding Standard for Java (2011) Chapter 2 - Input Validation and Data Sanitization (IDS)
MemberOf	C	864	2011 Top 25 - Insecure Interaction Between Components
MemberOf	C	875	CERT C++ Secure Coding Section 07 - Characters and Strings (STR)
MemberOf	C	878	CERT C++ Secure Coding Section 10 - Environment (ENV)
MemberOf	V	884	CWE Cross-section
MemberOf	C	929	OWASP Top Ten 2013 Category A1 - Injection
MemberOf	C	990	SFP Secondary Cluster: Tainted Input to Command
MemberOf	C	1027	OWASP Top Ten 2017 Category A1 - Injection
MemberOf	C	1131	CISQ Quality Measures (2016) - Security
MemberOf	C	1134	SEI CERT Oracle Secure Coding Standard for Java - Guidelines 00. Input Validation and Data Sanitization (IDS)
MemberOf	C	1165	SEI CERT C Coding Standard - Guidelines 10. Environment (ENV)

MemberOf	V	1200	Weaknesses in the 2019 CWE Top 25 Most Dangerous Software Errors
MemberOf	V	1337	Weaknesses in the 2021 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1347	OWASP Top Ten 2021 Category A03:2021 - Injection
MemberOf	V	1350	Weaknesses in the 2020 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1387	Weaknesses in the 2022 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1409	Comprehensive Categorization: Injection
MemberOf	V	1425	Weaknesses in the 2023 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1430	Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1435	Weaknesses in the 2025 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1440	OWASP Top Ten 2025 Category A05:2025 - Injection

Vulnerability Mapping Notes

Usage	**ALLOWED** *(this CWE ID may be used to map to real-world vulnerabilities)*
Reason	Acceptable-Use
Rationale	This CWE entry is at the Base level of abstraction, which is a preferred level of abstraction for mapping to the root causes of vulnerabilities.
Comments	Carefully read both the name and description to ensure that this mapping is an appropriate fit. Do not try to 'force' a mapping to a lower-level Base/Variant simply to comply with this preferred level of abstraction.

Notes

Terminology

The "OS command injection" phrase carries different meanings to different people. For some people, it only refers to cases in which the attacker injects command separators into arguments for an application-controlled program that is being invoked. For some people, it refers to any type of attack that can allow the attacker to execute OS commands of their own choosing. This usage could include untrusted search path weaknesses (CWE-426) that cause the application to find and execute an attacker-controlled program. Further complicating the issue is the case when argument injection (CWE-88) allows alternate command-line switches or options to be inserted into the command line, such as an "-exec" switch whose purpose may be to execute the subsequent argument as a command (this -exec switch exists in the UNIX "find" command, for example). In this latter case, however, CWE-88 could be regarded as the primary weakness in a chain with CWE-78.

Research Gap

More investigation is needed into the distinction between the OS command injection variants, including the role with argument injection (CWE-88). Equivalent distinctions may exist in other injection-related problems such as SQL injection.

Taxonomy Mappings

Mapped Taxonomy Name	Node ID	Fit	Mapped Node Name
PLOVER			OS Command Injection
OWASP Top Ten 2007	A3	CWE More Specific	Malicious File Execution
OWASP Top Ten 2004	A6	CWE More Specific	Injection Flaws
CERT C Secure Coding	ENV03-C		Sanitize the environment when invoking external programs
CERT C Secure Coding	ENV33-C	CWE More Specific	Do not call system()
CERT C Secure Coding	STR02-C		Sanitize data passed to complex subsystems

WASC	31		OS Commanding
The CERT Oracle Secure Coding Standard for Java (2011)	IDS07-J		Do not pass untrusted, unsanitized data to the Runtime.exec() method
Software Fault Patterns	SFP24		Tainted input to command
OMG ASCSM	ASCSM-CWE-78		

Related Attack Patterns

CAPEC-ID	Attack Pattern Name
CAPEC-108	Command Line Execution through SQL Injection
CAPEC-15	Command Delimiters
CAPEC-43	Exploiting Multiple Input Interpretation Layers
CAPEC-6	Argument Injection
CAPEC-88	OS Command Injection

References

[REF-140]	Greg Hoglund and Gary McGraw. *"Exploiting Software: How to Break Code".* Addison-Wesley. 2004-02-27. <https://www.amazon.com/Exploiting-Software-How-Break-Code/dp/0201786958>. (*URL validated: 2023-04-07*)
[REF-685]	Pascal Meunier. *"Meta-Character Vulnerabilities".* 2008-02-20. <https://web.archive.org/web/20100714032622/https://www.cs.purdue.edu/homes/cs390s/slides/week09.pdf>. (*URL validated: 2023-04-07*)
[REF-686]	Robert Auger. *"OS Commanding".* 2009-06. <http://projects.webappsec.org/w/page/13246950/OS%20Commanding>. (*URL validated: 2023-04-07*)
[REF-687]	Lincoln Stein and John Stewart. *"The World Wide Web Security FAQ".* chapter: "CGI Scripts". 2002-02-04. <https://www.w3.org/Security/Faq/wwwsf4.html>. (*URL validated: 2023-04-07*)
[REF-688]	Jordan Dimov, Cigital. *"Security Issues in Perl Scripts".* <https://www.cgisecurity.com/lib/sips.html>. (*URL validated: 2023-04-07*)
[REF-44]	Michael Howard, David LeBlanc and John Viega. *"24 Deadly Sins of Software Security".* "Sin 10: Command Injection." Page 171. McGraw-Hill. 2010.
[REF-690]	Frank Kim. *"Top 25 Series - Rank 9 - OS Command Injection".* SANS Software Security Institute. 2010-02-24. <https://www.sans.org/blog/top-25-series-rank-9-os-command-injection/>. (*URL validated: 2023-04-07*)
[REF-45]	OWASP. *"OWASP Enterprise Security API (ESAPI) Project".* <https://owasp.org/www-project-enterprise-security-api/>. (*URL validated: 2025-07-24*)
[REF-76]	Sean Barnum and Michael Gegick. *"Least Privilege".* 2005-09-14. <https://web.archive.org/web/20211209014121/https://www.cisa.gov/uscert/bsi/articles/knowledge/principles/least-privilege>. (*URL validated: 2023-04-07*)
[REF-62]	Mark Dowd, John McDonald and Justin Schuh. *"The Art of Software Security Assessment".* Chapter 8, "Shell Metacharacters", Page 425. 1st Edition. Addison Wesley. 2006.
[REF-962]	Object Management Group (OMG). *"Automated Source Code Security Measure (ASCSM)".* ASCSM-CWE-78. 2016-01. <http://www.omg.org/spec/ASCSM/1.0/>.
[REF-1449]	Cybersecurity and Infrastructure Security Agency. *"Secure by Design Alert: Eliminating OS Command Injection Vulnerabilities".* 2024-07-10.

	<https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-os-command-injection-vulnerabilities>. (*URL validated: 2024-07-14*)
[REF-1479]	Gregory Larsen, E. Kenneth Hong Fong, David A. Wheeler and Rama S. Moorthy. *"State-of-the-Art Resources (SOAR) for Software Vulnerability Detection, Test, and Evaluation"*. 2014-07. <https://www.ida.org/-/media/feature/publications/s/st/stateoftheart-resources-soar-for-software-vulnerability-detection-test-and-evaluation/p-5061.ashx>. (*URL validated: 2025-09-05*)
[REF-1481]	D3FEND. *"D3FEND: Application Layer Firewall"*. <https://d3fend.mitre.org/dao/artifact/d3f:ApplicationLayerFirewall/>. (*URL validated: 2025-09-06*)

Content History

Submissions

Submission Date	Submitter	Organization
2006-07-19 *(CWE Draft 3, 2006-07-19)*	PLOVER	

Contributions

Contribution Date	Contributor	Organization
2024-02-29 *(CWE 4.15, 2024-07-16)*	Abhi Balakrishnan *Provided diagram to improve CWE usability*	

Modifications

Previous Entry Names